

JavaScript

Advanced Concepts — Complete Study Guide

undefined · Scope · Lexical Environment · Scope Chain

var / let / const · Hoisting · Block Scope · Closures · setTimeout

Includes concepts, examples & interview Q&A for every topic

1 Understanding undefined in JavaScript

What is undefined?

undefined is a special keyword in JavaScript — a placeholder value automatically assigned to variables that have been declared in memory but not yet given a value. It is not empty, not null, and not an error. It simply means: memory allocated, no value yet.

Memory Allocation & Execution Flow

- JS allocates memory for **all variables** during Phase 1 (Memory Creation) before any line runs.
- At this point every variable starts as **undefined**.
- Phase 2 (Code Execution) replaces **undefined** with actual values line by line.

```
var a;  
console.log(a); // undefined (declared, memory allocated, no value)  
a = 7;  
console.log(a); // 7 (value assigned in Phase 2)  
  
console.log(x); // ReferenceError: x is not defined (never declared)
```

undefined vs not defined

Term	Meaning	Example	Error?
undefined	Variable declared, memory allocated, no value assigned yet	var a; console.log(a);	No — prints undefined
not defined	Variable never declared — no memory allocated at all	console.log(x);	Yes — ReferenceError

JavaScript is Loosely Typed

- Variables in JS have **no fixed data type** — they can change type at any time.
- This is why JS is called a **loosely typed** or **weakly typed** language.

```
var a;  
console.log(a); // undefined  
a = 10;  
console.log(a); // 10  
a = "hello world";  
console.log(a); // "hello world" — same variable, different type!
```

Checking for undefined

```
if (a === undefined) {  
  console.log("a is undefined");  
} else {  
  console.log("a has a value");  
}  
  
// Always use === (strict equality) — never ==
```

Best Practice: Never manually assign **undefined** to a variable (e.g. **a = undefined**). Let JavaScript handle it naturally. Manual assignment causes confusion and bugs.

Interview Q&A — undefined

Q1.

What is undefined in JavaScript?

■ A special placeholder value automatically assigned by JS to variables that are declared but not yet given a value during Phase 1 (memory creation).

Q2.

What is the difference between undefined and not defined?

■ undefined means the variable exists in memory but has no value. not defined means the variable was never declared — accessing it throws a ReferenceError.

Q3.

Why should you not manually assign undefined to a variable?

■ Because undefined is JS's signal that a variable is uninitialized. Manually assigning it creates confusion — you can't tell if it was set by the system or the developer.

Q4.

What does 'loosely typed' mean in JavaScript?

■ Variables are not bound to a specific data type. The same variable can hold a number, then a string, then a boolean — the type is determined at runtime.

2 Scope, Lexical Environment & Scope Chain

Core Definitions

Term	Definition
Scope	Where a variable or function can be accessed in code.
Lexical Environment	Local memory of the current EC + a reference to its parent's lexical environment.
Lexical Parent	The function/environment where a function is physically written in the source code.
Scope Chain	The linked chain of lexical environments used to look up variables, from innermost to outermost.

How the Scope Chain Works

- When a variable is accessed, JS first looks in the **local memory** of the current EC.
- If not found, it follows the **parent reference** to the outer lexical environment.
- This continues up the chain until the **global environment** is reached.
- If the variable is still not found → **ReferenceError**.

```
let b = 10; // global

function A() {
  // b is not declared here, so JS looks up the scope chain
  console.log(b); // 10 – found in global scope

  function C() {
    // C's lexical parent is A; A's lexical parent is global
    console.log(b); // 10 – found in global via scope chain
  }
  C();
}

A();
```

Lexical Environment Table

Execution Context	Local Memory	Parent Reference
Global	b = 10, function A	null (no parent)
Function A	function C (local)	Points to Global
Function C	(nothing local)	Points to A's environment

Scope Chain Lookup — Step by Step

```
// What happens when C looks up variable b:
// Step 1: Look in C's local memory      → not found
// Step 2: Follow parent ref → A's memory → not found
// Step 3: Follow parent ref → Global    → b = 10 found!
// Output: 10
```

Variable Declared Inside vs Outside

```
let b = 10; // global b
```

```
function A() {
  let b = 50; // shadows global b inside A

  function C() {
    console.log(b); // 50 — finds b in A's environment first
  }
  C();
}

A();
console.log(b); // 10 — global b is unchanged
```

Foundation for Closures: When a nested function retains access to its lexical parent's environment even after the parent has finished executing — that is a **closure**. Understanding lexical environments is the key to understanding closures.

Interview Q&A — Scope & Lexical Environment

Q1.

What is scope in JavaScript?

■ Scope determines where variables and functions are accessible. JS has global scope, function scope, and block scope.

Q2.

What is a lexical environment?

■ It is the combination of the current function's local memory and a reference to its parent's lexical environment. Every execution context has one.

Q3.

What is the scope chain?

■ The linked chain of lexical environments. When a variable is not found locally, JS follows parent references up the chain until it finds the variable or reaches global scope.

Q4.

What happens if a variable is not found anywhere in the scope chain?

■ JavaScript throws a `ReferenceError` — the variable is 'not defined'.

Q5.

How does lexical scope differ from dynamic scope?

■ Lexical scope is determined by where a function is written in the source code. Dynamic scope would depend on where a function is called. JS uses lexical scope.

3 var, let, const & Hoisting in Detail

Hoisting — What It Really Means

Hoisting is **not** literally moving code to the top. It is the result of JS allocating memory for all declarations during **Phase 1 (Memory Creation)** before any code runs. The source code never moves — only memory is set up early.

Temporal Dead Zone (TDZ)

- The period between entering a scope and the point where a **let/const** variable's declaration is reached.
- During TDZ the variable **exists in memory** but is completely inaccessible.
- Any access during TDZ throws a **ReferenceError**.
- TDZ ends the moment the declaration line is executed.

Complete Comparison: var vs let vs const

Feature	var	let	const
Scope	Function / Global	Block	Block
Hoisted?	Yes — as undefined	Yes — TDZ	Yes — TDZ
Initial value	undefined	TDZ (inaccessible)	TDZ (inaccessible)
Before declaration	Returns undefined	ReferenceError	ReferenceError
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Attaches to window	Yes	No	No
Must init at decl	No	No	Yes

var — Hoisted as undefined

```
console.log(a); // undefined — var is hoisted, initialized to undefined
var a = 10;
console.log(a); // 10

// var is function-scoped, not block-scoped:
{
  var x = 5;
}
console.log(x); // 5 — leaks out of the block!
```

let — Block Scoped, TDZ

```
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;     // TDZ ends here
console.log(b); // 20

// Block scoped:
{
  let y = 10;
  console.log(y); // 10
}
console.log(y); // ReferenceError: y is not defined
```

const — Must Initialize, Cannot Reassign

```
const c = 30;
c = 40;          // TypeError: Assignment to constant variable

const d;         // SyntaxError: Missing initializer in const declaration

// const is block scoped too:
{
  const PI = 3.14;
}
console.log(PI); // ReferenceError
```

Redeclaration Rules

```
var name = "Alice";
var name = "Bob";    // OK — var allows redeclaration

let age = 25;
let age = 30;        // SyntaxError: Identifier 'age' has already been declared

const city = "Delhi";
const city = "Mumbai"; // SyntaxError
```

Global Object Attachment

```
var a = 1;
let b = 2;
const c = 3;

console.log(window.a); // 1 — var attaches to global window object
console.log(window.b); // undefined — let does NOT attach to window
console.log(window.c); // undefined — const does NOT attach to window
```

Best Practice: Always prefer **const** by default. Use **let** when you need to reassign. Avoid **var** — its function-scoping and hoisting behaviour causes subtle bugs.

Interview Q&A — var / let / const / Hoisting

Q1.

What is the Temporal Dead Zone (TDZ)?

■ The period between the start of a block scope and the let/const declaration line. The variable exists in memory but cannot be accessed. Any access throws a ReferenceError.

Q2.

Why does var not have a TDZ but let and const do?

■ var is initialized to undefined during hoisting (Phase 1). let and const are hoisted but left uninitialized — they remain in TDZ until their declaration line is reached in Phase 2.

Q3.

What is the difference between var and let scoping?

■ var is function-scoped — it leaks out of blocks like if/for. let is block-scoped — it stays inside the {} where it was declared.

Q4.

Can you redeclare a let variable?

■ No. Redeclaring a let or const variable in the same scope throws a `SyntaxError`. `var` allows redeclaration.

Q5.

Why doesn't let attach to the window object?

■ `let` and `const` are stored in a separate memory space (block scope), not on the global object. `var` declarations in global scope become properties of `window`.

4 Block Scope & Shadowing

What is a Block?

A **block** (compound statement) is a group of statements enclosed in curly braces `{ }`. It allows multiple statements to be used where JS syntax expects only one (e.g., inside if, for, while, or functions).

Block Scope — let and const

- **let** and **const** declared inside a block are **block-scoped** — they only exist within that block.
- **var** declared inside a block is **NOT** block-scoped — it leaks to the enclosing function or global scope.

```
{
  var a = 1;    // function / global scoped – accessible outside
  let b = 2;    // block scoped – only inside { }
  const c = 3;  // block scoped – only inside { }

  console.log(a); // 1
  console.log(b); // 2
  console.log(c); // 3
}

console.log(a); // 1 – var leaked out
console.log(b); // ReferenceError: b is not defined
console.log(c); // ReferenceError: c is not defined
```

Memory: Where Are They Stored?

Declaration	Stored In	Accessible From
var	Global / function memory space	Anywhere in the function or globally
let	Separate block memory space	Only inside the <code>{ }</code> block
const	Separate block memory space	Only inside the <code>{ }</code> block

Shadowing

- A variable inside a block can have the **same name** as one outside — the inner one **shadows** the outer.
- The outer variable is **not affected** — shadowing only applies inside the block.

```
let x = 10; // outer x

{
  let x = 20; // inner x – shadows outer x
  console.log(x); // 20 – inner x
}

console.log(x); // 10 – outer x unchanged
```

Illegal Shadowing

```
var a = 10;
{
  let a = 20; // OK – let can shadow var
}

let b = 10;
{
```

```
var b = 20; // SyntaxError: 'b' has already been declared
// var cannot shadow let in the same scope – it would try
// to redeclare b in the outer scope, which is illegal
}
```

Practical Use of Blocks

```
// Without block – variable leaks
var total = 0;
for (var i = 1; i <= 5; i++) {
  total += i;
}
console.log(i); // 6 – i leaked out of the loop!

// With let – clean, no leakage
let sum = 0;
for (let j = 1; j <= 5; j++) {
  sum += j;
}
console.log(j); // ReferenceError – j stayed inside the loop
```

Interview Q&A — Block Scope

Q1.

What is a block in JavaScript?

■ A group of zero or more statements enclosed in curly braces {}. Used to group statements wherever JS syntax expects a single statement.

Q2.

Why does let/const behave differently from var inside a block?

■ let and const are block-scoped — they are stored in a separate memory space for that block. var is function-scoped and not restricted to the block.

Q3.

What is variable shadowing?

■ When a variable inside an inner scope has the same name as one in an outer scope, the inner one shadows (hides) the outer one within that scope. The outer variable is unaffected.

Q4.

What is illegal shadowing?

■ Using var to shadow a let variable in an outer scope. Since var ignores block scope, it tries to redeclare the let variable in the outer scope — causing a SyntaxError.

5 Closures — The Most Important JS Concept

What is a Closure?

A **closure** is a function bundled together with its **lexical environment** — the scope in which it was created. It allows a function to **remember and access variables from its outer scope even after the outer function has finished executing**.

Core Components

Component	Explanation
Function	The inner function that forms the closure.
Lexical Environment	The variables from the outer scope that the function remembers.
State Persistence	Variables inside the closure persist between calls — they act like private state.

Basic Closure Example

```
function outer() {
  let count = 0;          // lexical environment variable

  return function inner() {
    count++;              // inner remembers count from outer
    console.log(count);
  };
}

const increment = outer(); // outer() finished, but count is NOT gone!
increment(); // 1
increment(); // 2
increment(); // 3
```

- Even after **outer()** has returned, **count** persists inside the closure.
- Each call to **increment()** uses the **same count** reference — not a copy.

Closure with Parameters

```
function multiplier(factor) {
  return function(number) {
    return number * factor; // factor is remembered via closure
  };
}

const double = multiplier(2);
const triple = multiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
// Each closure has its own 'factor' – separate lexical environments
```

Data Encapsulation with Closures

```
function createCounter() {
  let count = 0; // private – cannot be accessed from outside

  return {
```

```

    increment: function() { count++; },
    decrement: function() { count--; },
    getCount: function() { return count; }
  };
}

const counter = createCounter();
counter.increment();
counter.increment();
counter.increment();
counter.decrement();
console.log(counter.getCount()); // 2
console.log(count); // ReferenceError – count is truly private!

```

Closures Capture References, Not Values

```

function makeAdder(x) {
  return function(y) {
    return x + y; // x is a reference from closure
  };
}

const add5 = makeAdder(5);
const add10 = makeAdder(10);

console.log(add5(3)); // 8
console.log(add10(3)); // 13
// add5 and add10 each have their own x in their lexical environment

```

Why Closures Matter

Use Case	How Closures Help
Data encapsulation / privacy	Variables inside closures can't be accessed from outside.
Stateful functions	Functions remember state across multiple calls.
Callbacks / event handlers	Functions remember context when called asynchronously.
Currying / partial application	Build specialised functions from general ones.
Module pattern	Create private and public members without classes.

Interview Q&A — Closures

Q1.

Define a closure in JavaScript.

■ A closure is a function that retains access to its lexical environment — the variables of its outer scope — even after the outer function has finished executing.

Q2.

Do closures store a copy of variables or a reference?

■ A reference, not a copy. All closures sharing the same outer variable reference the same memory location.

Q3.

What is a practical use of closures?

■ Data encapsulation — creating private variables that can only be modified through specific functions, like a counter or a bank account balance.

Q4.

What is the relationship between closures and lexical scope?

■ Closures depend on lexical scope — a function's scope is determined by where it is written, not where it is called. The closure captures that lexical environment.

Q5.

Can closures cause memory leaks?

■ Yes. If a closure holds references to large objects that are no longer needed, those objects won't be garbage collected. Always be mindful of what a closure captures.

6 Closures with setTimeout — Classic Interview Problem

How setTimeout Works

setTimeout schedules a callback to run after a delay. It does **NOT** block execution. JavaScript continues running the rest of the code immediately, and the callback is placed in the queue to run after the delay expires and the call stack is empty.

```
console.log("Before");
setTimeout(function() {
  console.log("Inside timeout");
}, 2000);
console.log("After");

// Output order:
// Before
// After
// Inside timeout (after 2 seconds – non-blocking!)
```

The Classic Problem — Printing 1 to 5 with Delays

Naive Approach Using var — BROKEN

```
for (var i = 1; i <= 5; i++) {
  setTimeout(function() {
    console.log(i);
  }, i * 1000);
}

// Expected: 1, 2, 3, 4, 5
// Actual:   6, 6, 6, 6, 6  <-- all print 6 !!
```

Why Does This Happen?

- All 5 callbacks share the **same reference** to **i** (var is function-scoped — only one i exists).
- The loop finishes immediately, incrementing i to **6**.
- After 1s, 2s, 3s... the callbacks finally run, but by then i is already **6**.
- Every callback reads the **same i** = 6.

Fix 1 — Use let (Block Scope per Iteration)

```
for (let i = 1; i <= 5; i++) {
  setTimeout(function() {
    console.log(i);
  }, i * 1000);
}

// Output: 1 (after 1s)
//         2 (after 2s)
//         3 (after 3s)
//         4 (after 4s)
//         5 (after 5s)
```

- **let** creates a **new binding** for **i** on every loop iteration.
- Each callback closure captures its **own separate i**.

Fix 2 — Explicit Closure Function (works with var)

```
function close(i) {
  setTimeout(function() {
    console.log(i); // this i is local to close() – a new copy each call
  }, i * 1000);
}

for (var i = 1; i <= 5; i++) {
  close(i); // creates a new execution context with its own i
}

// Output: 1, 2, 3, 4, 5 (with delays) – CORRECT
```

- Each call to **close(i)** creates a **new execution context** with its own local **i**.
- The callback inside closes over **that local i** — not the loop variable.

Fix 3 — IIFE (Immediately Invoked Function Expression)

```
for (var i = 1; i <= 5; i++) {
  (function(i) { // IIFE creates a new scope immediately
    setTimeout(function() {
      console.log(i);
    }, i * 1000);
  })(i); // pass current i as argument
}

// Output: 1, 2, 3, 4, 5 – CORRECT
```

Side-by-Side Comparison

Approach	i shared?	Works?	Why
var in loop	Yes — one i	No	All callbacks see final i = 6
let in loop	No — new i per iteration	Yes	Block scope creates fresh binding each time
Closure function	No — local i per call	Yes	New execution context for each iteration
IIFE	No — local i per IIFE	Yes	Immediately creates new scope with current i

Interview Q&A — Closures & setTimeout

Q1.

What does setTimeout do to execution flow?

■ It is non-blocking. The callback is scheduled for later; JS continues executing the next lines immediately. The callback runs only after the delay AND the call stack is empty.

Q2.

Why does a for loop with var and setTimeout print the wrong value?

■ All setTimeout callbacks share the same var i reference. The loop finishes (i becomes 6) before any callback runs. So all callbacks read i = 6.

Q3.

How does let fix the setTimeout problem?

■ let creates a new block-scoped binding of i for each iteration. Each callback captures its own separate i — they no longer share one reference.

Q4.

How can you fix the var + setTimeout problem without using let?

■ Wrap setTimeout in a function and pass i as an argument. Each function call creates a new execution context with its own local i, forming a closure over that local copy.

Q5.

What is an IIFE and how does it help here?

■ An Immediately Invoked Function Expression is a function that is defined and called at the same time. It creates a new scope immediately, capturing the current value of i per iteration.

Q6.

Explain what a closure is in the context of the setTimeout problem.

■ Each callback is a closure — it remembers (closes over) the i from its lexical environment. With var there is only one i; with let/closure function there is a separate i per iteration.

★ Master Cheat Sheet — All Concepts at a Glance

Concept	One-Line Definition	Key Watch-out
undefined	Placeholder JS assigns to declared but uninitialized variables.	Never manually assign undefined.
not defined	Variable never declared — ReferenceError thrown.	Different from undefined!
Loosely typed	Variables have no fixed type — can hold any value.	Type coercion can cause surprises.
Scope	Where a variable/function is accessible in code.	Nested functions can access outer scope.
Lexical Environment	Local memory + parent reference.	Determines what a function can see.
Scope Chain	Chain of lexical environments searched for variable lookup.	Stops at global; throws error if not found.
var	Function-scoped, hoisted as undefined, attaches to window.	Leaks out of blocks — avoid it.
let	Block-scoped, TDZ, no window attachment.	Cannot access before declaration.
const	Block-scoped, TDZ, must initialize, cannot reassign.	Use by default; prefer over let/var.
TDZ	Period before let/const declaration — any access = ReferenceError.	Exists even though variable is 'hoisted'.
Block { }	Groups statements; let/const are contained inside.	var ignores block scope.
Shadowing	Inner variable hides outer variable of same name.	Illegal to shadow let with var.
Closure	Function + its lexical environment = access to outer vars after return.	Captures references, not values.
setTimeout + var	All callbacks share one var i — see final value.	Fix: use let or closure function.
setTimeout + let	Each iteration gets its own i — correct output.	Block scope saves the day.

Top 5 Interview Traps to Memorise:

1	undefined ≠ not defined	undefined means memory allocated, no value. not defined means no memory at all.
2	Hoisting is not code movement	It is Phase 1 memory allocation. Source code stays where it is.
3	Closures capture references	They do not copy values. All closures sharing a var see the same variable.
4	var + loop + setTimeout = bug	All callbacks share one var. Use let or a closure wrapper function.
5	const ≠ immutable	const prevents reassignment of the binding, but object/array contents can still be mutated.